EBIS Business Techschool

Tema 4 · Clase 4

APIs propias, FastAPI y workflows LLM

Del script con estado al servicio HTTP probado con Postman



Máster en Ingeniería de Automatización con IA Agéntica

Clase 4

Convertimos el workflow con estado de la Clase 3 en un servicio API.



✓ **Resultado de hoy**

Una API FastAPI que recibe eventos por caso, actualiza estado y devuelve operation + state.

! **IMPORTANTE**

HTTP y el LLM son stateless. La aplicación conserva estado.

Después de gestionar contexto en scripts, aprendemos a exponer la misma lógica como contrato HTTP consumible por frontend, Postman o sistemas externos.

Objetivos de aprendizaje de la sesión

Al terminar, debemos poder diseñar y probar una API LLM con estado operativo.

01 **Distinguir script y servicio**

Qué cambia cuando pasamos de ejecución local a endpoint HTTP.

02 **Diseñar contratos API**

Request, response, case_id, state, operation y errores.

03 **Exponer el workflow de Clase 3**

Recibir eventos, cargar sesión, llamar al LLM y aplicar reglas.

04 **Probar con Postman y Swagger**

Ejecutar varios turnos y observar cómo cambia el estado.

Entregable práctico: api_models.py · session_store.py · app.py · postman_collection.json · endpoints con state/trace

El cambio clave: de script batch a backend orientado a eventos

La lógica LLM deja de ejecutarse una vez; ahora responde a llamadas HTTP independientes.

Clase 1
Invocar modelo

Clase 2
Controlar salida

Clase 3
Controlar estado

Clase 4
Exponer API

Ahora llega un evento

Un frontend, Postman o sistema externo envía un mensaje/cambio mediante HTTP.

La API conserva estado

El endpoint carga el case_id, construye contexto y actualiza la sesión.

La respuesta es operación

No devolvemos solo texto: devolvemos assistant_message, state, operation y métricas.

El objetivo no es “usar FastAPI”, sino convertir el workflow LLM en un componente reutilizable y trazable.

EBIS Business Techschool

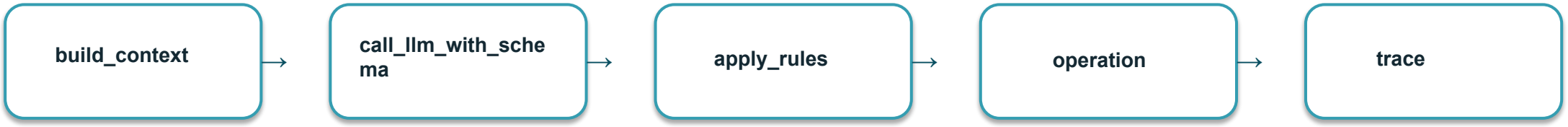
BLOQUE 1

1. De script a servicio

Qué cambia al exponer un workflow LLM como API.

Clase 3 nos dejó un workflow local con estado

Ahora lo encapsulamos detrás de un contrato HTTP.



Antes

python run_conversation.py
Lee CSV, itera turnos y guarda un trace al final.

Ahora

POST /cases/{case_id}/events
Recibe un evento y devuelve estado + operación.

Criterio

El mismo core lógico, otra interfaz de entrada y salida.

La API no sustituye a la lógica de Clase 3: la reutiliza y la hace consumible por otras aplicaciones.

HTTP es stateless. El LLM también. La aplicación no.

Cada request llega aislada, pero el backend reconstruye continuidad con case_id y state.

HTTP request

No recuerda la request anterior. Cada llamada debe traer identificador y payload.

LLM call

No conserva memoria real. Recibe contexto construido en cada llamada.

Application state

Guarda sesión, estado, resumen, trace y reglas deterministas.

request

```
POST /cases/S-001/events
{
  "message": "The order is 8921"
}
```

backend

```
case_id = "S-001"
state = store[case_id]
context = build_context(state, event)
```

La continuidad no está en el protocolo ni en el modelo: está en el diseño de la aplicación.

De CSV/script a endpoint: cambia la forma de entrada

Las peticiones sustituye al CSV como forma de simular eventos reales.

Script Clase 3	API Clase 4
Lee filas de CSV	Recibe JSON por HTTP
Itera todos los turnos	Procesa un evento por request
Guarda trace al final	Devuelve respuesta inmediata
Útil para batch/debug	Útil para frontend/integración

Consecuencia técnica

Necesitamos modelos de request/response, store de sesiones, control de errores y pruebas HTTP.

La API debe devolver más que una respuesta textual

Un cliente necesita saber qué ocurrió y qué debe hacer después.

request

```
{
  "event_type": "user_message",
  "message": "I want to return an item",
  "metadata": {"channel": "web"}
}
```

response

```
{
  "assistant_message": "Could you provide the order number?",
  "operation": "ask_for_order_id",
  "state": {...},
  "metrics": {...}
}
```

State

Foto operativa actual del caso.

Operation

Siguiente acción que debe ejecutar el sistema.

Metrics

Latencia, tokens y modelo para trazabilidad.

Endpoint central: eventos sobre casos

El endpoint recibe lo que ha ocurrido; la API decide cómo evoluciona el estado.

endpoint

```
POST /cases/{case_id}/events


{
  "event_type": "user_message",
  "message": "It arrived broken"
}
```

Por qué “events”

No todo será chat: también puede llegar un email, un comentario de operador o un campo manual.

Por qué “case_id”

Agrupa llamadas independientes bajo la misma instancia de workflow.

Por qué “operation”

El backend devuelve qué toca hacer, no solo texto para mostrar.

El endpoint recibe eventos; las operaciones son resultado del workflow, no necesariamente endpoints separados.

EBIS Business Techschool

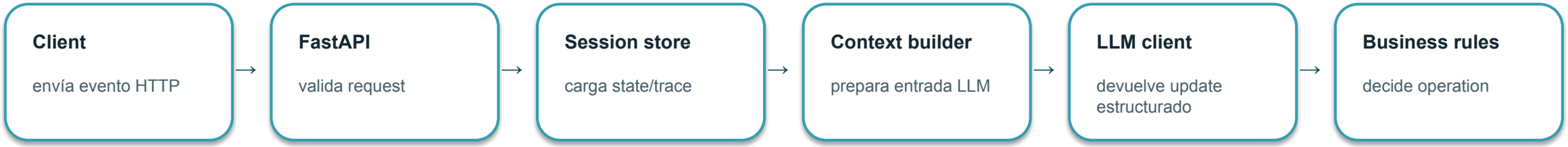
BLOQUE 2

2. Diseño de una API LLM con estado

Request/response, sesiones, trace y operaciones como contrato.

Arquitectura del servicio

El endpoint coordina; el modelo interpreta; Python valida y enruta.



Validación de entrada

Pydantic valida el payload antes de tocar lógica de negocio.

Salida estructurada

Reutilizamos tool/schema para recibir ConversationUpdate validado.

Respuesta estable

El cliente recibe assistant_message, operation, state y métricas.

Request y response son parte del producto

La API se gobierna con modelos Pydantic, no con diccionarios sueltos.

api_models.py

```python
class EventRequest(BaseModel):
    event_type: str = "user_message"
    message: str
    metadata: dict = Field(default_factory=dict)


class EventResponse(BaseModel):
    case_id: str
    assistant_message: str
    operation: str
    state: ConversationState
    metrics: RequestMetrics
```

Entrada controlada

No aceptamos cualquier JSON: definimos qué campos son válidos.

Salida predecible

El frontend no parsea texto: consume una estructura de dato.

Documentación automática

FastAPI genera OpenAPI/Swagger desde estos modelos.

case_id es el eje de continuidad entre requests

Cada llamada HTTP es independiente, pero apunta a la misma instancia de workflow.

same case_id

```
POST /cases/S-001/events
{ "message": "I want to return an item" }

POST /cases/S-001/events
{ "message": "The order is 8921" }

GET /cases/S-001/state
```

La store recupera sesión

sessions[case_id] devuelve messages, summary, state y trace.

El contexto se reconstruye

El endpoint ensambla reglas + estado + resumen + nuevo evento.

El cliente ve continuidad

Aunque HTTP no recuerde, la API responde como flujo coherente.

En producción el store no sería memoria local: sería Redis, PostgreSQL, MongoDB o una tabla de casos.

El endpoint no “pregunta al modelo”: procesa un evento

El modelo es una etapa dentro de una transacción de negocio más amplia.

1. **validate request**
2. **load session**
3. **build context**
4. **call LLM**
5. **apply rules**
6. **return response**

LLM stage

Interpreta el evento y propone un updated_state estructurado.

Application stage

Valida, calcula missing_fields, decide operation y guarda trace.

La API debe ser más determinista que el modelo: controla el ciclo de vida completo de la request.

Routing como resultado, no como endpoint separado

No creamos un endpoint por operación; devolvemos la operación que corresponde.

Mala lectura	Patrón recomendado
POST /ask_order_id	POST /cases/{case_id}/events → operation=ask_for_order_id
POST /start_refund_validation	POST /cases/{case_id}/events → operation=start_refund_validation
POST /send_to_human_review	POST /cases/{case_id}/events → operation=send_to_human_review

Por qué

La operación depende del estado completo del caso, no solo del mensaje actual.

Qué gana el diseño

La API mantiene una única entrada de eventos y un contrato de salida coherente.

Trazabilidad: state, trace y métricas en cada request

Una API LLM debe poder explicar qué pasó en cada turno.

trace event

```
{
  "case_id": "S-001",
  "event": "The order is 8921",
  "operation": "ask_for_reason",
  "state": {...},
  "metrics": {
    "latency_ms": 1840,
    "input_tokens": 920
  }
}
```

Debug

Ver qué contexto y estado produjo cada operación.

Evaluación

Comprobar si el estado converge correctamente.

Observabilidad

Preparar latencia, tokens, errores y routing para Clase 5.

EBIS Business Techschool

BLOQUE 3

3. FastAPI y HTTP

Cómo probar el workflow como servicio y manejar errores reales.

Endpoint principal: POST /cases/{case_id}/events

Una llamada HTTP actualiza el workflow y devuelve una operación.

app.py

```python
@app.post("/cases/{case_id}/events")
def process_event(case_id: str, request: EventRequest):
    session = get_or_create_session(case_id)
    prompt = build_context_prompt(session, request.message, config)
    result = call_llm_with_schema(...)
    state = apply_business_rules(result.object.updated_state)
    operation = determine_next_operation(state)
    return EventResponse(...)
```

Entrada

case_id + EventRequest

Proceso

session → context → LLM → rules → trace

Salida

EventResponse con state, operation y metrics

Endpoints de soporte: inspeccionar y resetear el caso

No todo endpoint debe llamar al modelo.

Endpoint	Responsabilidad	LLM call
GET /health	Comprobar que el servicio arranca	No
GET /cases/{case_id}/state	Consultar estado actual	No
GET /cases/{case_id}/trace	Consultar eventos y decisiones	No
DELETE /cases/{case_id}	Eliminar sesión local	No
POST /cases/{case_id}/events	Procesar nuevo evento	Sí

Idea clave

Separar endpoints de lectura/gestión de los endpoints que invocan el modelo.

Coste

Solo el endpoint de eventos debería generar coste LLM.

Postman: probar el flujo como usuario o sistema externo

Cada request avanza el mismo case_id y modifica el estado.

turn 1

```
POST /cases/S-001/events
{
  "message": "I want to return an item"
}

→ operation: ask_for_order_id
```

turn 2

```
POST /cases/S-001/events
{
  "message": "The order is 8921"
}

→ operation: ask_for_reason
```

turn 3

```
POST /cases/S-001/events
{
  "message": "It arrived broken"
}

→ operation: start_refund_validation
```

Tres llamadas HTTP independientes convergen a un estado completo.

FastAPI genera documentación viva del contrato

Swagger ayuda a probar.

local run

```
uvicorn app:app --reload

http://127.0.0.1:8000/docs
```

OpenAPI

Documenta endpoints, schemas, ejemplos y errores esperados.

Frontend

Consume la misma respuesta JSON sin saber cómo funciona el LLM.

Errores esperables en una API LLM

El servicio debe fallar de forma controlada y útil para el cliente.

Situación	HTTP	Respuesta segura
Payload inválido	422	Pydantic validation error
Mensaje vacío	400	Rechazar input
API key ausente	500	Config error controlado
Timeout proveedor/ fallo operative del prov	504/503/429	Retry/fallback o error explícito
Salida estructurada inválida	502	No avanzar workflow

Principio

Un error del modelo no debe corromper el estado del caso.

Buenas prácticas

Validar input, capturar excepciones y registrar trace con status.

Latencia y métricas por request

Cada endpoint LLM debe devolver o registrar metadatos operativos.

response metadata

```
"metrics": {
  "model": "claude-haiku-4-5",
  "latency_ms": 1840,
  "input_tokens": 920,
  "output_tokens": 140
}
```

Latencia

Impacta UX y throughput del backend.

Tokens

Explican coste y tamaño de contexto.

Modelo

Permite comparar versiones y depurar cambios.

Streaming: útil para UX, no siempre para workflows estructurados

La entrega incremental mejora experiencia, pero complica validación.

Cuándo sí

Respuestas largas, chat visible, generación de texto para usuario.

Cuándo no

Outputs JSON críticos, routing, operaciones que requieren validación completa.

optional

```
@app.get("/demo/stream")
def stream_demo():
    return StreamingResponse(token_generator(), media_type="text/plain")
```

EBIS Business Techschool

BLOQUE 4

4. Demo y práctica guiada

Construimos la API de eventos sobre el workflow de Clase 3.

Caso práctico: AI Request Triage API

Convertimos una conversación multi-turn en una API de eventos con estado.

Entrada

Eventos HTTP con message, event_type y metadata.

Estado

ConversationState por case_id.

Decisión

operation: ask, execute, review.

Salida

JSON estable + metrics.

input event

```
POST /cases/S-001/events
{
  "message": "I was charged twice for order 9901"
}
```

API response

```
{
  "operation": "send_to_human_review",
  "state": {"priority": "high", ...}
}
```

Estructura recomendada del repositorio

Reutilizamos el core de Clase 3 y añadimos la capa API.

project

```
class_04_fastapi_llm_service/
├── .env
├── config.yml
├── app.py
├── api_models.py
├── session_store.py
├── llm_client.py
├── schemas.py
├── prompts.py
├── business_rules.py
└── session_manager.py
```

Nuevo

app.py, api_models.py, session_store.py.

Reutilizado

schemas.py, prompts.py, business_rules.py, llm_client.py.

Resultado

API local ejecutable con uvicorn .

Construcción de la práctica: de core LLM a API

El objetivo es entender responsabilidades, no copiar un backend enorme.

A **api_models.py**

contrato HTTP

B **session_store.py**

estado en memoria

C **app.py**

endpoints

D **Swagger**

pruebas

E **errors**

fallo controlado

Paso 1 · api_models.py: contrato de entrada y salida

La API expone modelos pequeños, estables y documentables.

api_models.py

```python
class EventRequest(BaseModel):
    event_type: str = "user_message"
    message: str
    metadata: dict = Field(default_factory=dict)


class RequestMetrics(BaseModel):
    model: str
    latency_ms: int
    input_tokens: int
    output_tokens: int


class EventResponse(BaseModel):
    case_id: str
    assistant_message: str
    operation: str
    state: ConversationState
    metrics: RequestMetrics
```

Qué decide

Cómo se comunica el cliente con la API.

TODO

Añadir validaciones de message y metadata.

Criterio

Ningún endpoint debería devolver texto sin contrato.

Paso 2 · session_store.py: memoria local por case_id

Para clase usamos memoria en proceso; en producción sería una base persistente.

session_store.py

```
sessions: dict[str, SessionManager] = {}

def get_or_create_session(case_id: str):
    # TODO: create SessionManager if missing
    ...

def get_session(case_id: str):
    # TODO: return existing session or raise 404
    ...

def delete_session(case_id: str):
    # TODO: remove session from store
    ...
```

Responsabilidad

Separar almacenamiento de sesiones de los endpoints.

Limitación

Si reinicias el servidor, se pierde el estado.

Producción

Redis / DB / tabla de casos con TTL y auditoría.

Paso 3 · app.py: inicializar FastAPI y dependencias

El servicio carga config, entorno, cliente LLM y aplicación FastAPI.

app.py setup

```
with open("config.yml", "r", encoding="utf-8") as f:
    config = yaml.safe_load(f)


load_dotenv()
client = Anthropic(
    api_key=os.getenv("LLM_API_KEY"),
    timeout=int(os.getenv("LLM_TIMEOUT_SECONDS", 30)),
)


app = FastAPI(title="AI Request Triage API")
```

Qué aporta

Punto de entrada único para todos los endpoints.

TODO

Comprobar API key/model y preparar health check.

Patrón

Config y secretos separados como en Clase 1.

Paso 4 · endpoint de eventos: el core del servicio

El endpoint reutiliza el workflow de Clase 3 por cada request.

app.py

```python
@app.post("/cases/{case_id}/events")
def process_event(case_id: str, request: EventRequest):
    session = get_or_create_session(case_id)
    prompt = build_context_prompt(session, request.message, config)


    result = call_llm_with_schema(
        client=client,
        prompt=prompt,
        schema=ConversationUpdate,
        ...
    )


    # TODO: apply rules, update session and return response
```

Entrada

case_id + EventRequest.

TODO

Actualizar session.messages, session.state y trace.

Salida

EventResponse con operation + state + metrics.

Paso 5 · endpoints de estado, traza y reset

Leer estado no debería llamar al modelo ni generar coste.

app.py

```python
@app.get("/cases/{case_id}/state")
def get_case_state(case_id: str):
    session = get_session(case_id)
    return session.state


@app.get("/cases/{case_id}/trace")
def get_case_trace(case_id: str):
    # TODO: return session trace
    ...


@app.delete("/cases/{case_id}")
def reset_case(case_id: str):
    # TODO: delete session
    ...
```

Diagnóstico

GET /state permite ver si el workflow converge.

Auditoría

GET /trace permite revisar decisiones y métricas.

Pruebas

DELETE permite reiniciar un caso desde Postman.

Paso 6 · manejo de errores: proteger el estado

Si el LLM falla, el caso no debe quedar a medias.

app.py

```
try:
    result = call_llm_with_schema(...)
except TimeoutError:
    raise HTTPException(status_code=504, detail="LLM timeout")
except ValidationError:
    raise HTTPException(status_code=502, detail="Invalid structured output")
except Exception:
    raise HTTPException(status_code=500, detail="Internal error")
```

Regla

No actualices state hasta tener output validado.

TODO

Devolver errores claros sin exponer secretos ni stack traces.

Clase 5

Esto será base de logging, métricas y alertas.

Paso 7 · streaming demo: separado del workflow principal

Útil para enseñar UX incremental sin mezclarlo con routing estructurado.

streaming demo

```
from fastapi.responses import StreamingResponse


def token_generator():
    for chunk in ["Processing", " case", "...", " done"]:
        yield chunk


@app.get("/demo/stream")
def stream_demo():
    return StreamingResponse(token_generator(), media_type="text/plain")
```

Objetivo

Ver cómo FastAPI puede devolver datos incrementalmente.

Cuidado

No usarlo para decidir operation hasta validar salida completa.

Práctica

Endpoint opcional; el core de eventos sigue siendo estructurado.

Output: respuesta API con estado y operación

EventResponse

```
{
  "case_id": "S-001",
  "assistant_message": "What is the reason for the return?",
  "operation": "ask_for_reason",
  "state": {
    "process_type": "refund_request",
    "order_id": "8921",
    "reason": null,
    "missing_fields": ["reason"],
    "ready_for_next_step": false
  },
  "metrics": {"latency_ms": 1540}
}
```

Para frontend

Mostrar assistant_message y decidir UI según operation.

Para backend

Ejecutar routing o escalar según operation.

Para observabilidad

Registrar métricas y trace para análisis posterior.

Debrief: APIs LLM como workflows stateful

El endpoint no es una llamada al modelo: es una operación de negocio controlada.

1 HTTP stateless

Cada request llega aislada.

2 App stateful

case_id recupera estado y trace.

3 Contrato API

Request/response con Pydantic.

4 Salida estructurada

El LLM devuelve update validable.

5 Reglas en código

routing y seguridad no dependen solo del prompt.

6 Postman/OpenAPI

La API se puede probar y compartir.

Una API LLM robusta recibe eventos, actualiza estado, aplica reglas y devuelve la siguiente operación del workflow.

a la Clase 5

En la próxima sesión operaremos, evaluaremos y desplegaremos este servicio.

Clase 1

Cliente LLM

Clase 2

Salida estructurada

Clase 3

Estado/contexto

Clase 4

API propia

Clase 5

LLMOps

hoy

```
POST /cases/{case_id}/events
→ update state
→ return operation
→ trace metrics
```

Siguiente paso

Versionar prompts/modelos, evaluar calidad, registrar métricas, monitorizar errores y preparar despliegue controlado.

Meta: una API LLM no se considera lista hasta que puede evaluarse, observarse y operar con seguridad.